



EVENTHUB:

“A Django-Based Event Management, Ticketing and Recommendation platform”



Student ID: 25122859

Submission Date: 2nd October 2026

Submitted By:
Yogesh Dhakal

Submitted To:
Mr Umesh Regmi

Table of Contents

Executive Summary	3
1. Introduction	3
1.1 Background	3
1.2 Problem Statement	3
1.3 Purpose and Scope	4
2. Aim, and Research Questions	4
2.1 Aim	4
2.2 Research Questions	4
3. System Requirements	4
3.1 Functional Requirements	4
3.2 Non-Functional Requirements	5
4. Methodology and Data Preparation	5
4.1 Development Approach	5
4.2 Dataset Sources and Composition	5
4.3 Data Preparation	5
4.4 Recommendation Methods	6
4.4.1 Rule-Based Recommendation	6
4.4.2 Content-Based Recommendation	6
4.4.3 Trending Recommendation	6
4.5 Machine Learning Method	6
4.6 Evaluation Approach	6
4.7 Tools and Technologies	6
5. System Architecture and Database Design	7
5.1 Overall Architecture	7
5.2 Django Application Modules	7
5.3 Database Design	8
6. Implementation	8
6.1 Authentication and Role Management	8
6.2 Event and Ticket Management	8
6.3 Booking and Checkout	8
6.4 Saved Events and Onboarding	8
6.5 Recommendation Interface	9
6.6 Machine Learning Integration	9
6.7 Analytics Dashboards	9
6.8 Project Structure	9
7. Testing and Evaluation	9
7.1 Recommendation Evaluation	10
7.2 Machine Learning Results	10
7.3 Analytics Results	10
8. User Interface	11
9. Challenges, Limitations and Risks	14
9.1 Challenges Encountered	14
9.2 Project Limitations	14
10. Ethical, Fairness, Privacy and Security Considerations	15
10.1 Recommendation Bias and Diversity	15
10.2 Machine Learning Reliability	15
10.3 Privacy	15
10.4 Security Controls	15
11. Future Work	15
12. Conclusion	16
References	16

Executive Summary

EventHub is a Django based web app to unite event discovery, event administration, ticket management and booking activities all under one roof. Offers unique functionality for attendees, event organizers and administrators. The visitors have the possibility to look through and search for events, save events, book events and get recommendations. Organisers can create and manage events, set up different types of tickets, track registrations, and analyse the events. Admins for a platform can see the platform-level information.

The project implements 3 approaches for recommendations: Rule based, Content based (feature vectors & cosine similarity), trending (based on activity in last 14 days). A cold-start fallback is used for the users that are not having much interaction history. The platform also has a machine learning module which classifies event popularity as an event popularity prediction problem. Logistic Regression is used as baseline and Random Forest model as the final model. The available evaluation set has a reported ROC-AUC score of 0.800 for Random Forest.

The data used for development is a combination of real venue and category details from Hugging Face Seeker, 15 real Nepal events, and generated synthetic users, events and interactions. The synthetic records allow controlled testing, but make it difficult to be certain about the generalizability of the results to real users. Payment-selection interface elements are provided for Khalti and eSewa but it does not have payment processing capabilities.

EventHub is just viewed as an academic prototype showcasing web development, database design, recommendation technique, machine learning, insights, and application security. More work needs to be done before being ready to use in production, including larger scale evaluation, integration of live payment, deployment, and operational monitoring.

1. Introduction

1.1 Background

Event organisers can share event information and accept registrations using paper or a spreadsheet, social media, or messaging applications. As the number of events grows, ticket types expand and the number of attendees rises, these strategies can start to get tricky to manage. Event information, ticket availability and booking records can be centralized in a web-based event management system. Just listing events, however, doesn't always make it easier for users to know which events are significant to them.

EventHub solves the problems of event administration and event discovery. It has a variety of basic features like authentication, event creation, and ticket booking, as well as features for using user interests, event attributes, and recent engagement for recommendations. This is no ordinary create-read-update-delete (CRUD) app.

1.2 Problem Statement

Users will have to browse several listings before they find events that meet their criteria, and organisers will need to have effective tools to control events, ticket types, capacity, and registrations. When records are not connected, it can be more challenging to monitor bookings and ticket availability. A standard event listing and booking system might also not be equipped with personalised discovery and a recent event engagement overview.

The project explores the possibility of integrating these functions in a single Django application without losing clarity in the recommendation logic and with a prudent attitude towards the results of machine learning.

1.3 Purpose and Scope

The aim of EventHub is to develop, create and test a platform for event discovery and management, ticket booking, personalized recommendations, popularity prediction and analytics using the Django framework. This covers user registration and role management, organiser event and ticket administration, event browsing/search/filtering, saved events and bookings, recommendation and trending features, ML-based popularity prediction, and organiser/administrator dashboards.

Payment processing using Live Khalti/eSewa, large-scale production deployment and load testing are not implemented.

2. Aim and Research Questions

2.1 Aim

The main aim of eventhub is to design, develop, and evaluate a Django-based event management platform that combines event booking with recommendation functionality, event popularity prediction, and analytics.

2.2 Research Questions

1. How can user interests and event information be used to recommend relevant events?
2. How can content-based similarity identify events similar to those with which a user has interacted?
3. How can recent event activity be represented through a transparent trending score?
4. How effectively can machine learning classify event popularity while reducing the risk of data leakage?
5. How can event management, recommendations, booking, analytics, and machine learning be integrated within a Django application?

3. System Requirements

3.1 Functional Requirements

ID	Area	Requirement	Status
FR1	Authentication	Users can register and log in.	Implemented
FR2	Roles	Users and organisers receive role-specific access.	Implemented
FR3	Event management	Organisers can create, update, and manage events.	Implemented
FR4	Ticket management	Events can contain ticket types and capacity information.	Implemented
FR5	Discovery	Users can browse, search, and filter events.	Implemented
FR6	Booking	Users can book available event tickets.	Implemented
FR7	Saved events	Users can save and revisit events.	Implemented
FR8	Recommendations	Users receive rule-based and content-based recommendations.	Implemented
FR9	Trending	The application displays events with recent engagement.	Implemented
FR10	Popularity prediction	Organisers can view ML-based popularity predictions.	Implemented
FR11	Analytics	Organisers and administrators can view dashboards.	Implemented
FR12	Payments	Checkout displays Khalti/eSewa payment-method choices.	Interface only

3.2 Non-Functional Requirements

Requirement	How it is addressed
Usability	Navigation, event cards, search and filtering, responsive layout, and an onboarding interests survey.
Security	Authentication, role checks, ownership checks, CSRF protection, and protected staff analytics.
Maintainability	Major functions are separated into Django applications.
Testability	Automated tests cover key authentication, event, booking, recommendation, and ML workflows.
Reliability	Booking capacity and recommendation fallback behaviour are tested.
Transparency	Recommendation explanations and trending calculations are designed to be understandable.
Scalability	The modular design provides a basis for future expansion; production-scale performance has not been established.

4. Methodology and Data Preparation

4.1 Development Approach

The development was done in an iterative and incremental fashion. The basic Django features were added first, followed by event, ticket workflows, data preparation, recommendation methods, machine learning, analytics, testing, and interface fixes and improvements. In this sequence individual features could be checked before the next sequence of functionality was added.

4.2 Dataset Sources and Composition

The project brings together a number of data sources instead of one large production dataset. Venue and category information were extracted from the Hugging Face dataset `rebrowser/seatgeek-dataset`, which was selected as a data source for the project.

There are also 15 real event records from Nepal in the application. Synthetic users, events and interaction records were created to have controlled examples for recommendation and model development. The synthetic interactions were not completely uniform random records, but were biased according to the interests of the users.

This combination allowed for the testing without a large EventHub production interaction database. However, the synthetic component needs to be considered a constraint: the resulting interactions could not reveal any real behavior of humans. The report thus separates out real event related data from generated test data and cannot say that the evaluation is a live marketplace.

4.3 Data Preparation

The data preparation workflow comprised an investigation of the available CSV and application data, extraction of features for the model, training and evaluation data separation. The machine learning notebook documents exploratory data analysis, feature engineering, model comparison, model evaluation, and saving the model. The recommendation components use event attributes and interaction records to recommend.

4.4 Recommendation Methods

4.4.1 Rule-Based Recommendation

The rule based recommender uses a set of attributes of the user and the event to generate a score. Scoring logic includes user interests, categories, popularity and interaction information. The method is clearly specified, which means that the application can give a clear explanation for the recommendation of an event. It also acts as a backup when the users have not had enough interactions.

4.4.2 Content-Based Recommendation

The recommender for the content-based approach extracts the information from the events and represents it as feature vector through DictVectorizer. Event representations are then compared by cosine similarity and events with similar characteristics are identified. It is not only about other users' behaviour, but also about attributes of items and interactions with them. If there are no or too few history items for a user, the system uses the rule-based recommendations.

4.4.3 Trending Recommendation

The trending component is based on a 14-day period and calculates an activity score. It is meant to showcase recent engagement and not just activities based on lifetime views. The calculation has been verified with the database: the score that was calculated manually was the same as in the application in the documented verification example.

4.5 Machine Learning Method

The problem of predicting the popularity of events is regarded as classification problem. Logistic Regression is used as a baseline model and Random Forest is evaluated as the final model. Constructed pipeline for inspecting, preprocessing, feature engineering, splitting into train and test set, training, evaluating, and saving the model. Views and registration count are not considered to be part of the features used in the model as they can directly contain or heavily encode the prediction target.

4.6 Evaluation Approach

The functionality for recommendations was tested both on application and database level, with fall back behaviour in the cold case and verification of the trending score. ROC-AUC was used to evaluate the machine-learning classifier. The Random Forest result is presented in Section 7.2.

4.7 Tools and Technologies

Layer	Technology / tool
Backend	Python and Django
Database	SQLite for development
Frontend	HTML, CSS, JavaScript, and Django templates
Machine learning	scikit-learn, Logistic Regression, and Random Forest
Recommendation	DictVectorizer, cosine similarity, and rule-based scoring
Data processing	pandas and CSV datasets
Development	Jupyter Notebook, Git, and GitHub
Testing	Django automated tests

5. System Architecture and Database Design

5.1 Overall Architecture

EventHub uses Django's Model–View–Template (MVT) architecture. A user's browser sends requests to Django views. Views apply application logic, use Django models to read or update database records, and render templates for the browser. Recommendation, machine learning, and analytics components process relevant data and return results for presentation through the interface.

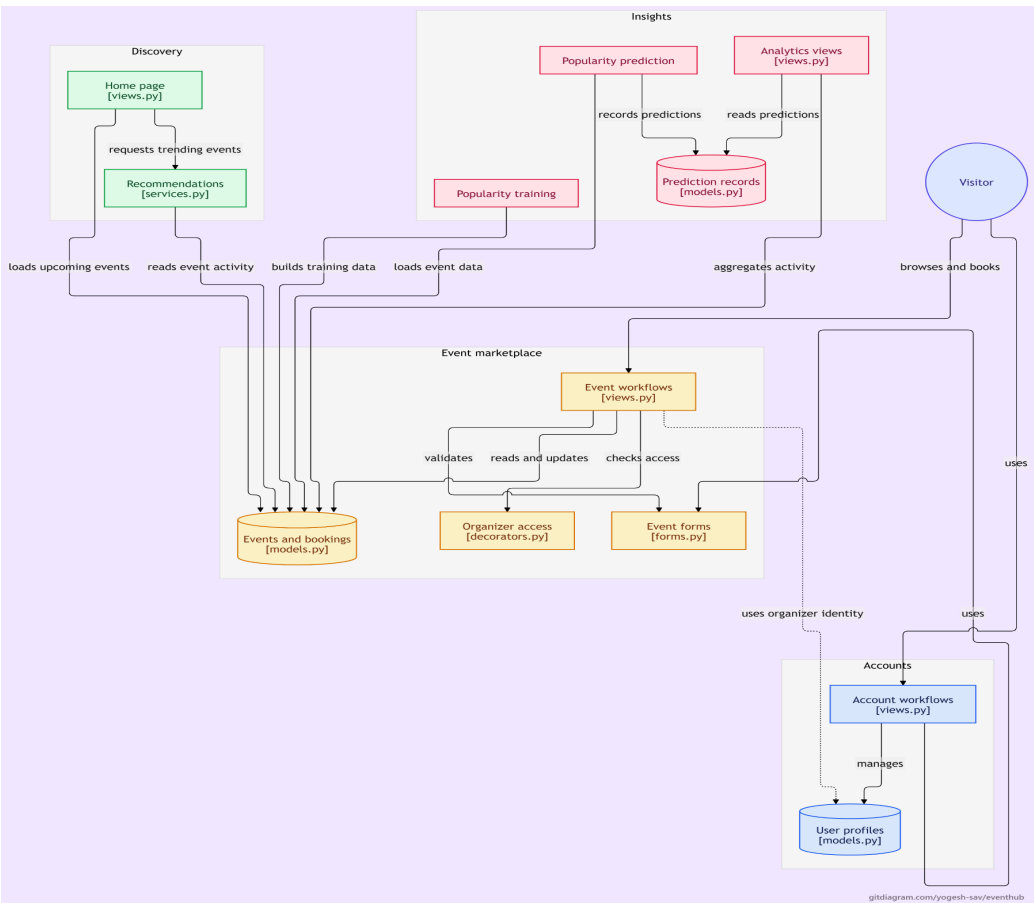


Figure 1. EventHub system architecture.

5.2 Django Application Modules

Module	Main responsibility
accounts	Registration, login, user profiles, and role information.
events	Event records, ticket types, saved events, bookings, and event interactions.
recommendations	Rule-based, content-based, cold-start fallback, and trending logic.
ml	Popularity prediction workflow, model artefacts, and prediction integration.
analytics	Organizer and administrator dashboards and summary information.
core	Shared project-level functionality and configuration.

5.3 Database Design

The main application entities include Profile, Event, TicketType, Booking, SavedEvent, EventInteraction, and PopularityPrediction. These entities represent the core data structures of EventHub and support user management, event administration, ticket booking, user interactions, and popularity prediction.

Entity	Purpose and relationships
User / Profile	Represents an account and its application role and interests. A user can create bookings, save events, and generate interaction records.
Event	Stores event information and is associated with its organiser. An event can have multiple ticket types and related interactions.
TicketType	Stores a ticket category, price, and capacity or availability information for an event.
Booking	Connects a user with an event and ticket information, recording a reservation and its quantity.
SavedEvent	Records an event that a user has saved for later.
EventInteraction	Records relevant user activity associated with an event and supports recommendation logic.
PopularityPrediction	Stores or represents an ML-based popularity prediction associated with an event.

6. Implementation

6.1 Authentication and Role Management

The accounts component manages registration, login, user profiles, and role information. The application distinguishes normal users from organisers and restricts organiser functions through authorization checks. Protected administrative analytics are intended for staff users; requests from non-staff users are rejected with a 403 response.

6.2 Event and Ticket Management

The events component allows organisers to create and manage event listings. An event can contain multiple ticket types, each with its own ticket information. Public users can browse, search, and filter events. Draft and cancelled events are excluded from normal public access. Event views also support interactions such as viewing, saving, and booking.

6.3 Booking and Checkout

The Booking model records reservations and ticket information. Capacity checks are used to prevent bookings from exceeding the available quantity. The checkout interface displays an order summary and payment-method choices. Khalti and eSewa are currently interface-level placeholders: no live payment API processing is implemented, and the report does not treat the payment flow as a completed integration.

6.4 Saved Events and Onboarding

Users can save events and view them later through a saved-events page. The onboarding interests survey is shown after registration, can be skipped, and stores selected interests in the user's profile. These interests help provide an initial recommendation signal before a substantial interaction history exists.

6.5 Recommendation Interface

The recommended-events page presents events selected by the recommendation components and displays an explanation of why an event has been recommended. This explanation is intended to improve transparency. Rule-based fallback behaviour allows the page to provide recommendations for users with limited history.

6.6 Machine Learning Integration

The ML notebook contains exploratory analysis, feature engineering, model comparison, evaluation, and model saving. Logistic Regression is used as the baseline and Random Forest is used for the final popularity prediction implementation. Views and registration_count are excluded from the predictive features to reduce leakage risk. The organizer dashboard presents the resulting prediction information.

6.7 Analytics Dashboards

The organiser dashboard provides summary information, registration trends, event utilisation, and popularity predictions. The administrator dashboard provides platform-level totals, category information, engagement trends, and most-viewed event information. Access to the administrator dashboard is restricted to staff users.

6.8 Project Structure

The repository is organised into separate Django applications and supporting folders. The documented high-level structure is:

```
EventHub/
├── accounts/
├── analytics/
├── events/
├── recommendations/
├── ml/
├── core/
├── templates/
├── static/
├── dataset/
├── notebooks/
├── ml_training.ipynb
├── manage.py
└── project configuration files
```

7. Testing and Evaluation

Testing was carried out throughout development. The documented test suite covers authentication, event permissions and management, booking and capacity limits, saved events, recommendation fallback logic, trending calculations, machine learning preprocessing, leakage exclusion, and analytics access.

Test area	Coverage	Latest Result
Authentication	Registration and login behaviour.	Passed
Event management	Creation, editing, and permission checks.	Passed
Booking	Booking workflow and capacity limits.	Passed
Saved events	Save and toggle behaviour.	Passed
Recommendations	Fallback and recommendation branches.	Passed
Trending	Database-level score verification.	Passed
Machine learning	Preprocessing and leakage exclusion.	Passed
Analytics	Dashboard data and protected access.	Passed

The final automated test suite passes all 33 of 33 tests. The signup-role test fixture was updated to reflect the Terms and Conditions checkbox introduced in the registration flow, and the complete suite now passes. The organiser-role feature was separately reported as manually verified. This distinction is retained so that the report accurately reflects the recorded automated test status.

7.1 Recommendation Evaluation

The rule-based method provides recommendations based on user interests and event attributes, with an explanation for the recommendation. The content-based method supports similarity-based recommendations and falls back to the rule-based method when user history is unavailable. The trending component uses a 14-day activity window; a database-level manual recalculation matched the displayed score in the documented verification.

7.2 Machine Learning Results

Model	Role in experiment	Reported result
Logistic Regression	Baseline classifier	Used for comparison
Random Forest	Final popularity classifier	ROC-AUC = 0.800

The Random Forest model achieved a ROC-AUC of 0.800 on the available evaluation dataset. This result indicates the model's measured ability to distinguish the target classes on that particular test set. The dataset and test set are small, so the result should not be interpreted as proof of production performance or reliable generalisation to a wider event population. A larger independently collected dataset and cross-validation would be needed for stronger evaluation.

```

--- LogisticRegression ---
Accuracy: 0.697  F1: 0.286  ROC-AUC: 0.748
precision      recall    f1-score      support

not popular    0.70     0.95     0.81         22
popular        0.67     0.18     0.29         11

accuracy
macro avg      0.68     0.57     0.55         33
weighted avg   0.69     0.70     0.63         33

--- RandomForest ---
Accuracy: 0.667  F1: 0.267  ROC-AUC: 0.684
precision      recall    f1-score      support

not popular    0.69     0.91     0.78         22
popular        0.50     0.18     0.27         11

accuracy
macro avg      0.59     0.55     0.53         33
weighted avg   0.63     0.67     0.61         33

Selected model: LogisticRegression (ROC-AUC 0.748)

```

Figure 2: Machine learning evaluation output.

7.3 Analytics Results

The organiser dashboard turns booking and event records into registration and utilisation summaries. The administrator dashboard presents platform-level totals and engagement information. These features demonstrate how operational records can be summarised for organisers and staff. No claim is made that the dashboards have been validated under production-scale traffic.

8. User Interface

The final interface includes a responsive navigation bar, homepage hero area, trending and upcoming event sections, search, event cards, event details, booking and checkout pages, saved events, recommendations, onboarding interests, organiser analytics, administrator analytics, and informational pages such as About Us, Contact Us, and Terms and Conditions. Custom 403 and 404 pages are also included for controlled error presentation.

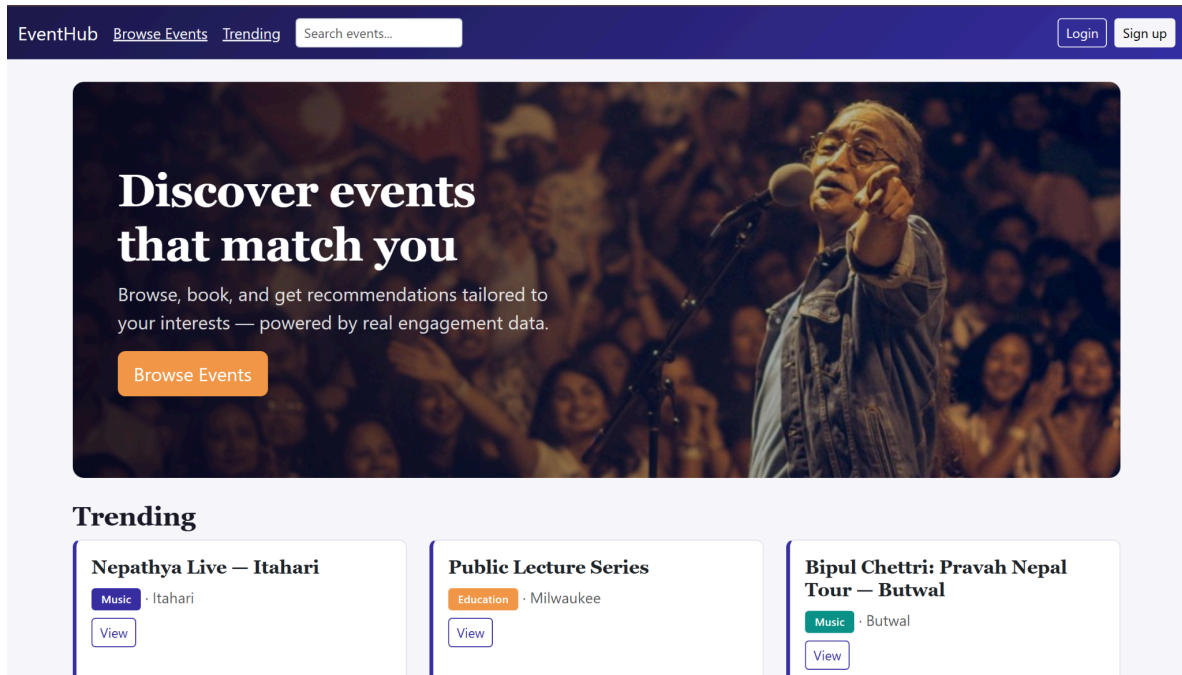


Figure 3: Home page.

The screenshot shows the EventHub sign-up page. It has the same dark blue navigation bar as the homepage. The main content area is titled 'Sign up' in a large, bold, black font. Below the title are four input fields: 'Username:', 'Email:', 'Password:', and 'Password confirmation:'. Below these fields are two radio buttons for 'Role:' with options 'User' and 'Organizer'. Below the radio buttons is a checkbox labeled 'I agree to the Terms & Conditions'. At the bottom of the form is a large orange button labeled 'Create account'. The footer of the page is dark blue and contains links for 'About Us', 'Contact Us', and 'Terms & Conditions', along with the copyright notice '© 2026 EventHub'.

Figure 4: Signup page

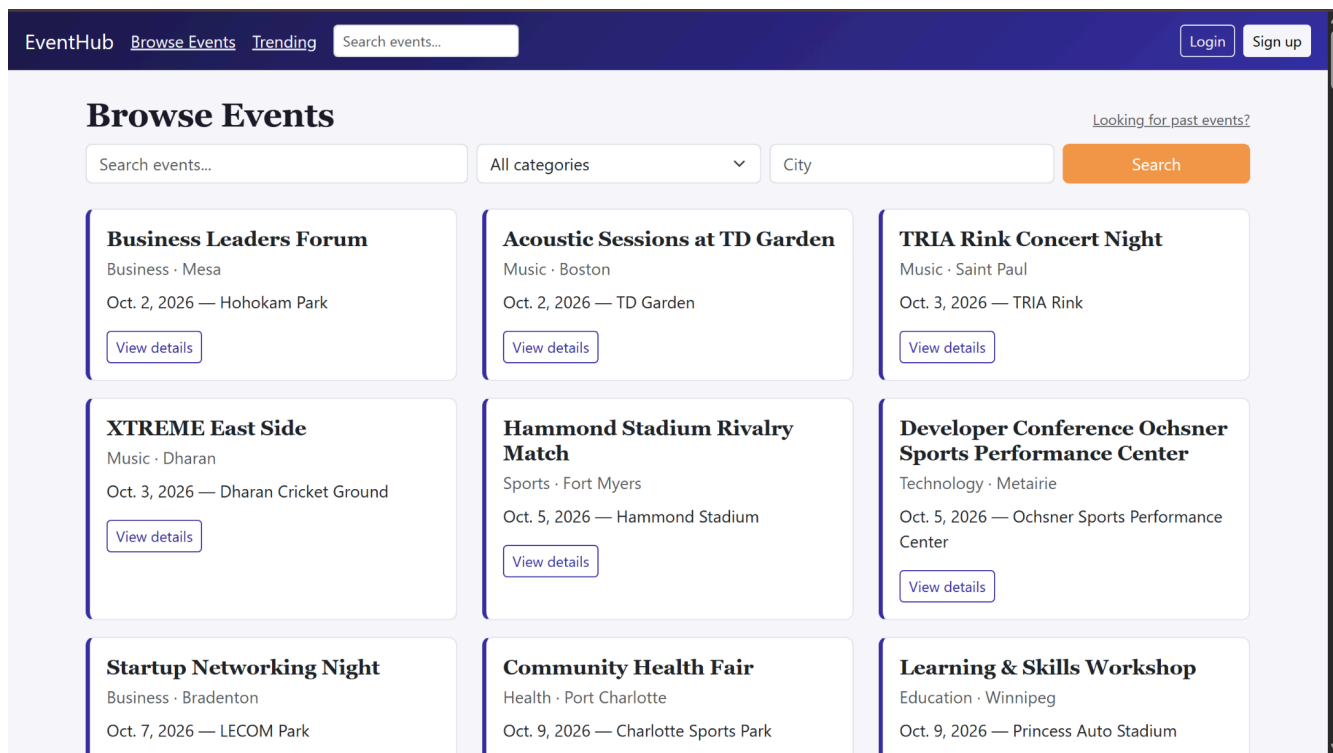


Figure 5: Event browsing and search.

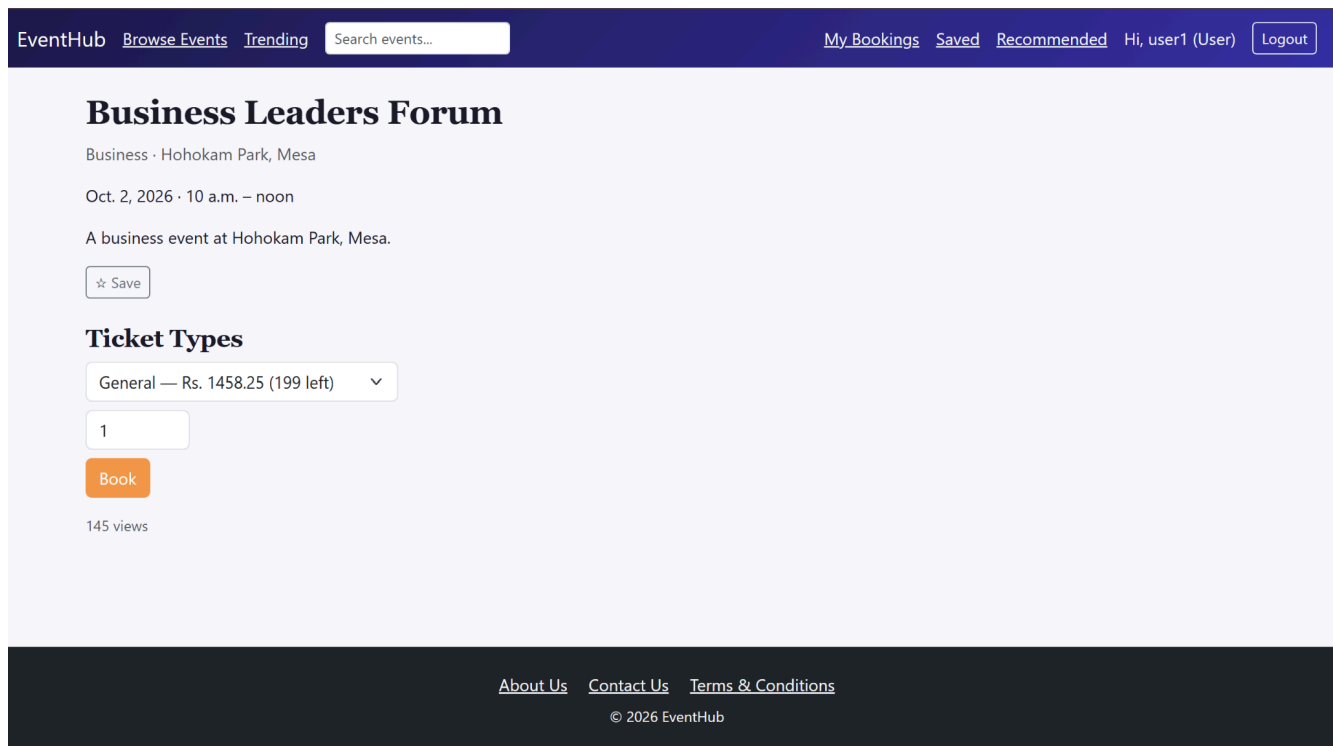


Figure 6: Event details and booking.

EventHub
Browse Events
Trending
My Bookings
Saved
Recommended
Hi, user1 (User)
Logout

Recommended for You

Art & Culture Fair

Art · Sarasota

Oct. 24, 2026 — Ed Smith Stadium

Why this event?
Similar to Art events you've engaged with

[View details](#)

XTREME East Side

Music · Dharan

Oct. 3, 2026 — Dharan Cricket Ground

Why this event?
Similar to Music events you've engaged with

[View details](#)

Gallery Opening Night

Art · Minneapolis

Nov. 21, 2026 — U.S. Bank Stadium

Why this event?
Similar to Art events you've engaged with

[View details](#)

TRIA Rink Concert Night

Music · Saint Paul

Oct. 3, 2026 — TRIA Rink

Why this event?
Similar to Music events you've engaged with

[View details](#)

Rewind Fest: Homecoming Dashain Edition

Music · Butwal

Oct. 10, 2026 — Butwal International Convention Center

Why this event?
Similar to Music events you've engaged with

[View details](#)

Public Lecture Series

Education · Milwaukee

Oct. 17, 2026 — Fiserv Forum

Why this event?
Similar to Education events you've engaged with

[View details](#)

Youth Music Fest

Gallery Opening Night

Godawari Mahotsav 2083

Figure 7: Recommended events.

EventHub
Browse Events
Trending
My Bookings
Saved
Recommended
Hi, organizer1 (Organizer)
My Events
Analytics
Logout

My Events

[+ Create Event](#)

Title	Date	Status	Views	Registrations	Popularity	
Prudential Center Concert Night	Sept. 10, 2026	Published	159	2	16% confidence	Edit Delete
Acoustic Sessions at CoolToday Park	Sept. 21, 2026	Published	223	3	46% confidence	Edit Delete
Django workshop	Oct. 10, 2026	Published	0	0	Not predicted yet	Edit Delete
Public Lecture Series	Oct. 17, 2026	Published	335	33	22% confidence	Edit Delete
Capital One Arena Rivalry Match	Oct. 21, 2026	Published	1	1	26% confidence	Edit Delete
Community Health Fair	Oct. 26, 2026	Published	105	1	28% confidence	Edit Delete
Team Member RSVP Rivalry Match	Oct. 26, 2026	Published	96	1	Likely popular (86%)	Edit Delete
LAN Party Night	Oct. 27, 2026	Published	118	2	Likely popular (83%)	Edit Delete
Public Lecture Series	Oct. 29, 2026	Published	122	2	18% confidence	Edit Delete
Career Development Seminar	Nov. 10, 2026	Published	160	2	24% confidence	Edit Delete
Learning & Skills Workshop	Dec. 10, 2026	Published	232	1	28% confidence	Edit Delete
LAN Party Night	Dec. 22, 2026	Published	73	3	Likely popular (64%)	Edit Delete
Learning & Skills Workshop	Dec. 24, 2026	Published	234	2	26% confidence	Edit Delete

Figure 8: Organizer's event page.

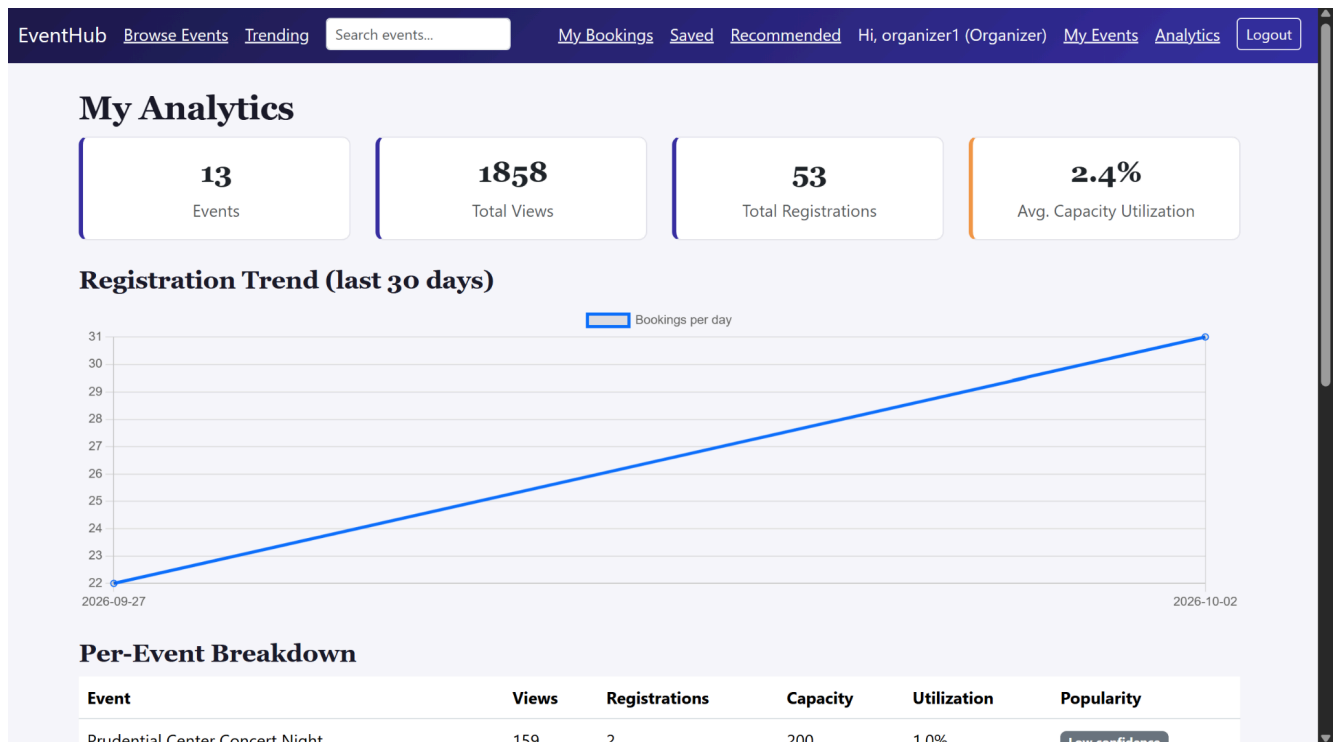


Figure 9: Organizer's analytics page.

9. Challenges, Limitations and Risks

9.1 Challenges Encountered

- Finding event datasets that were suitable for the project and available under usable access conditions.
- Combining real event-related information with controlled synthetic users and interaction records.
- Handling cold-start recommendations for users without interaction history.
- Reducing data leakage risk in the popularity prediction model.
- Maintaining recommendation behaviour while adding booking and interface features.
- Updating automated tests after changes to the registration interface.

9.2 Project Limitations

- The machine learning evaluation uses a relatively small test set; the ROC-AUC result requires validation on more data.
- Some users, events, and interactions are synthetic and may not represent real-world behaviour.
- Recommendations may concentrate around a narrow set of user interests and may not provide sufficient diversity.
- Khalti and eSewa are interface-level payment options, not live payment integrations.
- The application has not been evaluated under large-scale production traffic.
- Production deployment, monitoring, logging, backup procedures, and infrastructure hardening are outside the current academic implementation.

10. Ethical, Fairness, Privacy and Security Considerations

10.1 Recommendation Bias and Diversity

Recommendation systems can reproduce patterns present in the data used to generate recommendations. EventHub uses synthetic interactions and a limited set of event information, so the resulting recommendations may not represent the full range of attendee interests or event categories. A production version should monitor category exposure and consider diversity constraints so that recommendations do not become overly concentrated.

10.2 Machine Learning Reliability

The popularity model is an academic prediction component. Its ROC-AUC of 0.800 is specific to the available project evaluation data. It should not be treated as a guarantee of future event popularity or as a substitute for organiser judgement. Larger datasets, cross-validation, and monitoring would be needed before operational use.

10.3 Privacy

EventHub uses user-related information, including interests, bookings, saved events, and interactions, to provide application functions and recommendations. A production deployment should provide clear privacy notices, limit data collection to appropriate purposes, protect stored information, define retention periods, and follow applicable data protection requirements.

10.4 Security Controls

- Authentication is used to protect user accounts.
- Role-based access checks restrict organiser and administrator functionality.
- Ownership checks limit event-management actions to authorised organisers.
- Django CSRF protection is used for form submissions.
- Administrative analytics are restricted to staff users.
- Sensitive configuration values should be supplied through environment-based settings rather than committed to source control.
- Custom 403 and 404 pages provide controlled error responses.

11. Future Work

- Integrate and test live Khalti payment processing, including secure verification and transaction handling.
- Integrate live eSewa payment processing with appropriate callback and verification handling.
- Introduce additional ticket categories such as Student and VIP tickets, with clearly defined discount rules.
- Collect a larger, consent-aware real-world event interaction dataset.
- Apply k-fold cross-validation and evaluate the model on a larger independent test set.
- Investigate collaborative filtering and hybrid recommendation approaches.
- Introduce recommendation-diversity constraints and evaluate category coverage.
- Improve the explainability and calibration of popularity predictions.
- Migrate from the development SQLite database to a production database such as PostgreSQL where appropriate.
- Deploy the application to a cloud environment and add monitoring, logging, backups, and operational security controls.

12. Conclusion

EventHub delivers a Django-based event management platform that combines event discovery, organiser event administration, ticket types, bookings, saved events, recommendations, machine learning, and analytics. It provides role-specific functionality for attendees, organisers, and administrators and brings the main event and booking workflows into one application.

The recommendation functionality combines explainable rule-based scoring, content-based similarity using DictVectorizer and cosine similarity, and a recent-activity trending method based on a 14-day window. A cold-start fallback allows recommendations to remain available when interaction history is limited. The machine learning component compares Logistic Regression with Random Forest for popularity classification; the Random Forest model achieved a ROC-AUC of 0.800 on the available test data. Views and registration count were excluded from predictive features to reduce data leakage.

The project demonstrates the integration of web development, database modelling, recommendation methods, machine learning, analytics, security, and testing. Its findings must be interpreted within the limitations of synthetic interaction data and a small evaluation set. Live payment processing and production deployment remain future work. Within its academic scope, EventHub provides a working prototype and a foundation for further development.

References

Jeny, J.R.V., Sadhana, P., Kumar, B.J., Abhishek, S.L. and Chander, T.S. (2022) ‘A web based-college event management system and notification sender’. In: *2022 4th International Conference on Inventive Research in Computing Applications (ICIRCA)*, pp. 1434–1438. Available at: <https://doi.org/10.1109/ICIRCA54612.2022.9985774>.

Nugroho, M.A. and Fachrie, M. (2024) ‘Pengembangan aplikasi penjualan tiket acara berbasis website dan mobile’, *Jurnal Indonesia: Manajemen Informatika dan Komunikasi*, 5(1), pp. 746–756. Available at: <https://doi.org/10.35870/jimik.v5i1.552>.

Yudhanto, Y., Pratisto, E.H., Purnomo, F.A., Hidayat, T.N., Haqimi, N.A. and Ardhi, O.D.W. (2024) ‘Design and development museum ticketing system (MTS) with design thinking method’, *Procedia Computer Science*, 234, pp. 1212–1219. Available at: <https://doi.org/10.1016/j.procs.2024.03.117>.

Django Software Foundation (n.d.) *Django documentation*. Available at: <https://docs.djangoproject.com/> [Accessed 2 October 2026].

scikit-learn developers (n.d.) *scikit-learn documentation*. Available at: <https://scikit-learn.org/stable/> [Accessed 1 October 2026].

Hugging Face (n.d.) *rebrowser/seatgeek-dataset*. Available at: <https://huggingface.co/datasets/rebrowser/seatgeek-dataset> [Accessed 1 October 2026].

Youtube Video link: <https://youtu.be/YXAMeb8AQnY>